构建知识闭环：用CodeBuddy打造自我进化的数据分析体系

原创 腾讯程序员 腾讯技术工程 2025年12月5日 01:31 日本 318人



作者：hassonlin、davidpzxie

难以查找海量库表、需求紧急复杂、重复性工作多、结果校验困难、追溯记录困难……数据分析时要如何破除这些痛点？QQ浏览器的信息流推荐架构团队基于CodeBuddy AI给出了迅速有效的答案。

1.引言

我们团队的部分职责是承担推荐数据分析的工作。在长期数据分析中，我们总结出以下业务痛点：

- 涉及到大量库表和口径，查找起来有困难；
- 需求紧急而复杂，仅有一人负责大部分工作，压力较大；
- 不同需求往往用到类似的口径，重复度较高；
- 数据分析对结果准确性要求极高，校验压力大；
- 代码和分析结果散落在各处，容易丢失，难以回溯。

我们的解决方案： 在有限的人力下快速试错、改进，构建了一个**知识闭环体系**。这个体系通过"**让知识可沉淀** → **让知识可传递** → **让知识可生成** → **让知识可信赖** → **让知识可闭环**"五个环节，形成了一个自我进化的良性循环。在此流程中，我们利用CodeBuddy AI来设计开发规范，还使用它的自定义Agent功能自动生成SQL代码，将编码过程从小时级下降到分钟级。

第一环：让知识可沉淀。 我们使用Git仓库管理所有数据分析代码和口径定义，使所有历史任务沉淀为可复用的知识库。

第二环：让知识可传递。 为了降低新负责人的学习成本，我们将代码目录结构化管理，并设定好代码规范。这样，原本只有一人负责的工作能方便地扩展到组内，让更多同学共同开发和维护。

第三环：让知识可生成。 为了降低数据分析过程中的重复工作量，我们利用CodeBuddy快速设计出一个数据分析Agent，它能够自动读取仓库内的历史代码，然后结合当前需求写出高质

量的分析SQL。面对简单和中等复杂的情境，生成代码的准确率能达到80%以上；而对于复杂情境，在1~2次运行和反馈后，代码也普遍能达到满意效果。

第四环：让知识可信赖。 虽然代码编写重要，但后续对运行**结果准确性的验证更重要且更有挑战**。这要求我们在对业务指标具备认知的基础上，再运用AI进行逻辑复查。

第五环：让知识可闭环。 由于数据分析需求杂乱，此前的代码和分析结果往往散落在各处，导致需要修改周期重新运行或回看结果时难以查找。为此，在利用Git的版本控制能力的基础上，我们还设定了文件命名规范，并要求在代码文件中记录分析结果的链接，以保障所有结果可溯。更重要的是，经过验证的新口径、新代码会重新进入Git仓库，成为知识库的一部分，供后续任务复用。

总结来说，重构后的数据分析流程有如下表格所示的优势，是一件值得参考的AI实践案例。

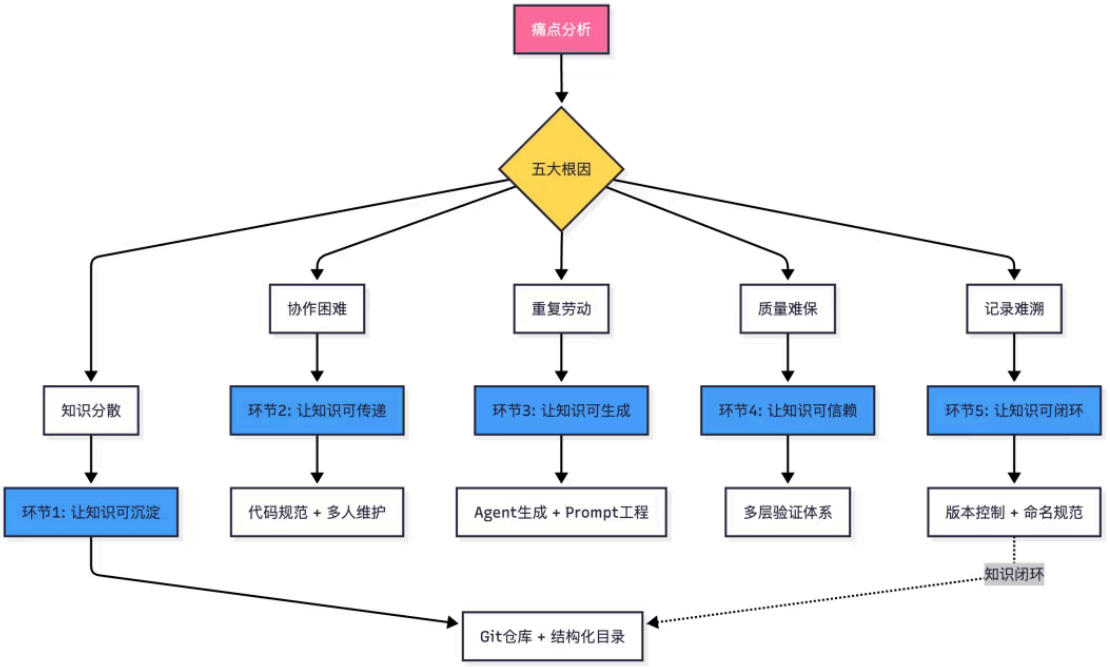
	重构前	重构后
数据口径	数量巨大，难以查找	分类整理，可RAG检索
负责人数	一人负责，压力山大	多人负责，轮班工作
重复性工作	石器时代，手工完成	AI生成，一键搞定
结果校验	人工复核，效率低下	AI辅助，人工为主
追溯工作记录	大海捞针，苦不堪言	规范记录，随时找到

知识闭环的价值： 这套体系最大的价值在于形成了正向循环。**每次数据分析任务都在为知识库添砖加瓦，知识库越丰富，AI生成的代码就越准确**，团队效率就越高。

在完成流程重构后，我们一方面反思教训、总结经验，认识到AI在代码生成上存在能力边界；另一方面，也展望通过AI自动化生成文档的优势，来攻克知识传递瓶颈，提升团队效率。

2.重构方法论：五环闭环体系

在解决这些痛点时，我们总结出一套可复用的方法论。这套方法论的核心思路是：**构建一个知识自我进化的闭环体系**。



这套体系由五个相互关联的环节组成，每个环节都承担着特定的职责。第一环是**让知识可沉淀**，通过Git管理代码并建立结构化目录来构建知识库，这样所有历史任务都能成为可复用的资产。第二环是**让知识可传递**，我们制定了统一的代码规范和文件注释标准来实现规范化协作，使得多人协作成为可能而不再依赖单个人。第三环是**让知识可生成**，利用Agent和Prompt工程实现AI自动化，提升编码效率。第四环是**让知识可信赖**，通过建立多层验证体系来保障质量。第五环是**让知识可闭环**，依靠版本控制和命名规范实现闭环增值，让经过验证的新知识重新沉淀回知识库，从而推动知识持续增长并形成正向循环。

通过这次实践，我们发现有五点特别重要：

1. **基础工作不能跳过**：没有Git知识库，AI就没有学习材料；没有代码规范，AI生成的代码质量就没保障
2. **人的流程优先**：先把人的协作流程搞顺畅，再用AI来加速，而不是一上来就用AI
3. **验证是核心**：AI生成代码只是第一步，验证准确性才是关键价值
4. **闭环是灵魂**：只有经过验证的知识重新沉淀，才能形成正向循环
5. **边做边改**：每个环节都要快速试一试，根据实际效果来调整方向

2.1.一个真实案例

接下来，我们先通过一个真实案例直观感受效果，然后再详细讲解每一步的实践过程。

需求： 筛选2025年9月21-27日期间关注过特定puin的一线城市用户，查询其在浮层场景的内容消费历史。

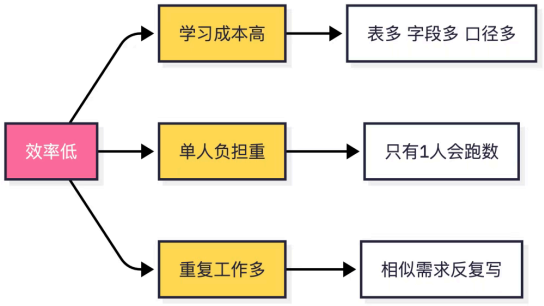
这个需求在重构前后的处理方式有着天壤之别。在重构前，整个流程需要花费30分钟翻阅各种文档来找到准确的口径定义，然后花费数小时手工编写复杂的多表关联逻辑，接着完全依赖人工检查来验证结果准确性，最后还要花10分钟手动记录各种信息到不同的文档和文件夹中，整个过程耗时且容易出错。

而在重构后，只需要2分钟就能让CodeBuddy自动检索Git仓库中的口径定义，然后10分钟就能生成符合团队规范的SQL代码，虽然验证环节仍然需要人工业务判断作为核心，但现在有了CodeBuddy辅助进行逻辑复查，最后借助Git版本控制只需1分钟就能完成归档工作，整个流程变得高效且规范。

下面，让我们详细讲解这套方案背后的工程实践以及细节思考。

3.构建知识库- 让知识可沉淀

通过分析我们发现，效率低下的根源主要体现在三个方面：首先是**学习成本高**，因为涉及大量的表、字段和口径定义，每次都需要花费大量时间去查找和理解；其次是**单人负担重**，由于只有一个人掌握数据分析技能，所有需求都压在一个人身上，工作压力巨大；最后是**重复工作多**，相似的需求需要反复编写类似的代码，大量时间浪费在重复劳动上。



3.1.规范管理代码

基于前面的分析，我们选择Git来管理所有数据分析代码。相比传统的在线文档或数据分析平台，Git有三个核心优势：

1. **代码即知识**：SQL代码本身就是最精确的知识表达，一行代码胜过千言万语
2. **版本可追溯**：天然支持版本控制，每次修改都有记录，为后续的知识闭环打下基础
3. **大模型可解析**：结构化的代码比自然语言更适合大模型理解和生成，维护成本更低

核心目标： 让每次数据分析任务都成为可复用的知识资产，而不是一次性的工作。

3.2.仓库结构设计

为了让Git仓库发挥好知识库的作用，我们需要对目录进行结构化管理。

在设计目录结构时，我们首先思考了一个问题：数据分析任务中哪些内容是可以复用的？经过分析发现，虽然每次分析需求都不同，但背后用到的数据口径往往是相似的，比如"一线城市用户"的定义、"浮层场景"的判断逻辑、"内容消费"的统计口径等，这些口径会在不同的分析任务中反复使用。基于这个洞察，我们确定了核心设计思路：**将可复用的口径定义与具体的分析代码分离管理，让口径成为独立的知识资产。**

分离"分析代码"和"口径定义"：

- **分析代码：**存所有历史任务的完整代码（如实验分析、异常兜底策略分析）
- **口径定义**：保存这些任务用到的数据口径（如实验桶号、用户消费数据）

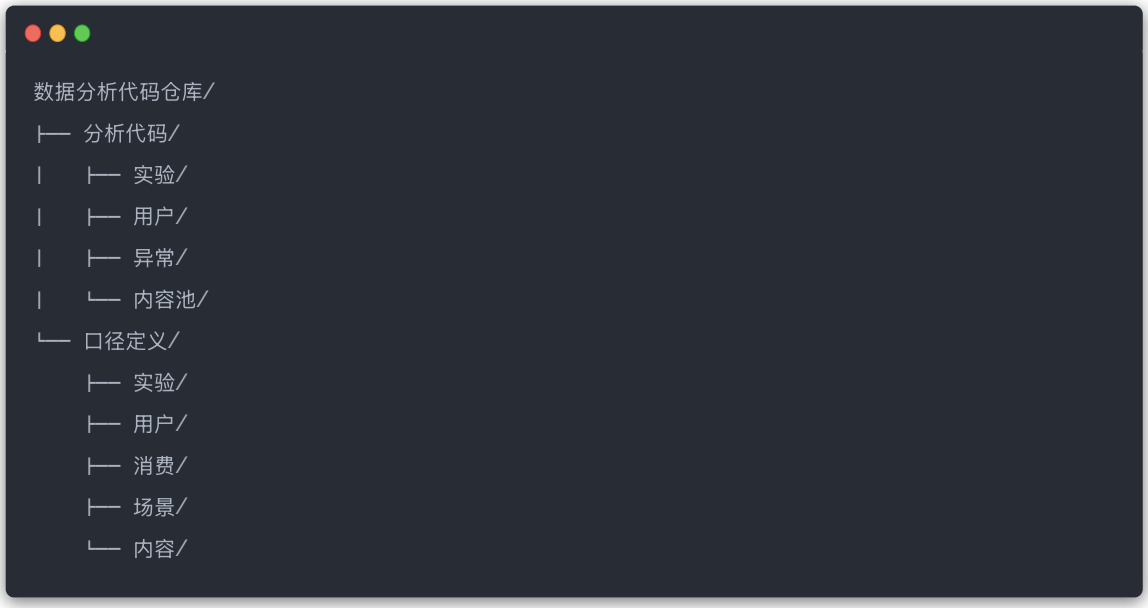
为什么要分离？

1.口径可以跨任务复用，避免重复定义

2.AI可以先检索口径，再生成分析代码

3.新人可以快速了解有哪些可用口径

目录结构示例：



实践经验：

- **目录结构可以用大模型辅助设计，再手动调整**
- 根据业务特点调整
- 遇到新口径时，及时更新到口径定义目录
- 随着需求迭代，口径库会越来越完善，数据分析效率越来越高

通过Git仓库和结构化目录，我们成功建立了知识沉淀的基础设施。但仅有知识库还不够，我们需要让这些知识能够在团队中有效传递，让更多人能够理解和使用这些沉淀下来的代码。

腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
690篇原创内容
公众号

4.制定规范 - 让知识可传递

有了Git仓库作为知识库，下一步是让这些知识能够在团队中有效传递。我们发现，知识传递的核心障碍不是找不到代码，而是看不懂代码和不知道怎么用。要让知识真正在团队中流动起来，需要建立一套完整的规范体系和协作机制。在规范生成过程中，CodeBuddy生成了最重要的初版内容，后续再由我们手动调整，使格式更加精炼易读。

4.1.为什么需要规范

在引入规范之前，虽然我们已经有了Git仓库，但团队协作仍然困难重重。每个人的代码风格不同，新人看历史代码如同天书，相似的逻辑在不同文件中写法各异，导致学习成本居高不下。更严重的是，当我们尝试让AI基于这些代码生成新代码时，由于缺乏统一规范，AI生成的代码质量参差不齐，有时甚至无法运行。这让我们意识到，必须建立统一的代码规范，让知识以标准化的形式沉淀和传递。

4.2.使用CTE

我们团队统一要求使用CTE（公用表表达式）编写数据分析代码，避免多层嵌套的SELECT语句。

我们之所以强制要求使用CTE，是因为在实践中发现**AI生成嵌套子查询时经常出现括号层级错误**，导致SQL无法正常执行。统一使用CTE格式后，AI生成的**SQL语法错误明显减少，代码的可读性和可维护性也得到了显著提升**。不过需要注意的是，**每个CTE都应该有明确的业务含义**，避免为了使用CTE而过度拆分逻辑。

在实际应用中，我们建议**在Prompt中明确要求使用CTE**，并提供具体的代码示例让AI学习正确的格式。对于AI生成场景来说，**CTE比嵌套子查询更可控、更易维护**，这不仅降低了语法错误的概率，也让后续的代码审查和优化工作变得更加轻松。

```
WITH
-- [CTE功能说明]
cte_name1 AS (
    SELECT ...
    FROM ...
    WHERE ...
),
-- [CTE功能说明]
cte_name2 AS (
    SELECT ...
    FROM ...
    WHERE ...
)
-- 结果输出
SELECT `[中文字段1]`, `[中文字段2]`
FROM ...
WHERE ...
```

4.3.文件注释规范：让代码自解释

数据分析代码往往行数较多（200行以上），且由于SQL的语法特性，其执行逻辑和自然语言逻辑差距较大。为了在不深入代码细节的前提下也能理解代码逻辑，我们决定在所有分析代码和口径定义文件的开头都加上规范的注释。

分析代码的注释格式如下所示，包含了**任务名、用户范围、时间范围、场景限制、数据源、分析任务、产出结构和结果链接**。

```
/*
 * [分析任务名称]
 *
 * 【用户范围】
 *   - [用户圈定条件]
 *
 * 【时间范围】
 *   - [具体日期或时间周期]
 *
 * 【场景限制】
 *   - [场景说明]
 *
 * 【数据源】
 *   - [表名]（[表定义]）
 *   - [表名]（[表定义]）
 *
 * 【分析任务】
 *   - [任务1]
 *   - [任务2]
 *
 * 【产出结构】
 *   - [字段1]：[定义]（[数据类型]）
 *   - [字段2]：[定义]（[数据类型]）
 *
 * 【结果链接】
 *   - 待补充
 */
```

口径定义的注释格式如下所示，包含了**口径名、功能、使用方法、筛选条件和输出字段**。

```
/*
 * [口径名称]
 *
 * 【功能】
 *   - [一句话简要描述功能]
 *
 * 【使用方法】
 *   - [需要关联什么字段来引用此CTE]
 *
```

```
 * 【筛选条件】
 *   - ${参数1}：[定义]，格式如 [格式说明]（[数据类型]）
 *   - ${参数2}：[定义]，格式如 [格式说明]（[数据类型]）
 *
 * 【输出字段】
 *   - [字段1]：[定义]（[数据类型]）
 *   - [字段2]：[定义]（[数据类型]）
 */
```

4.4.协作流程规范

为了防止不规范的代码污染知识库，我们建立了两项核心机制：主分支保护，所有修改必须通过feature分支提交；强制代码审查，至少一人review通过后才能合并到主分支，确保每一次提交都符合团队规范。

通过这一步，我们让知识真正在团队中流动起来，**从单人负责变成多人协作，从个人经验变成团队资产**。现在我们有多位同学共同数据分析，并基于Git进行版本管理，当某位同学处理新分析需求或新口径定义时，可以将代码便捷地推送至主分支，便于其他同学在后续数据分析任务中复用。接下来，我们需要让AI基于这些沉淀的知识自动生成新代码，进一步提升效率。

5.设计Agent - 让知识可生成

在完成知识沉淀和知识传递后，我们终于可以让AI基于这些知识自动生成新代码，从而大幅降低重复工作量。我们利用CodeBuddy快速设计出一个数据分析Agent，它能够自动读取仓库内的历史代码，然后结合当前需求写出高质量的分析SQL。

核心目标： 让AI基于已沉淀的知识自动生成新代码，实现效率的指数级提升。

重要提示： 这一步只是知识闭环的开始，AI生成的代码需要经过验证和沉淀，才能成为新的知识资产。

5.1.自定义Agent

在技术选型的过程中，我们考虑过自行部署Text2SQL框架（如vanna）和司内的一些AI数据分析平台（如dola），但这些方案或者**试错成本较高**，或者自身不够成熟。基于**敏捷开发**原则，我们优先选择试错成本最低的方案来快速验证想法，最终决定使用CodeBuddy来创建一个能理解代码仓库并生成SQL代码的Agent。

这个决策基于四个关键考虑：首先，CodeBuddy与我们日常使用的JetBrains IDE深度集成，作为后端开发团队，我们本来就在IDE中编写和管理代码，使用CodeBuddy可以在熟悉的环境中完成从需求分析到代码生成的全流程，无需切换工具，**学习成本几乎为零**；其次，我们已经有Git仓库作为知识库，CodeBuddy可以直接读取和理解这些代码，天然**契合我们的工作方式**；再次，小范围试用后效果超出预期，面对简单和中等复杂的数据分析需求，生成代码的准确率能达到80%以上，证明了这个方案的**可行性**；最后，即使这个方案最终不够理想，我们在Git仓库中沉淀下来的代码规范、口径定义和历史分析代码，也可以无缝迁移到其他

Text2SQL框架或数据分析平台，这些知识资产不会因为工具切换而浪费，这让我们**可以放心地快速试错。**

参考codebuddy文档，创建自定义Agent的方式十分简单，几步即可完成。我们将这个数据分析Agent命名为"SQL大王"，场景提示词填入下一节介绍的prompt内容，工具只需要选择"搜索文件"、"读取文件"、"搜索代码"和"编辑文件"即可，如下图所示。



提问时，建议选择编程能力最强的Claude-4.5-Sonnet模型。

5.2.Prompt设计

Prompt设定的好坏直接影响模型表现，我们参考同事的km文章，设计出了结构清晰的prompt。它由五个部分组成：**角色与任务、核心原则、执行步骤、输出格式和生成案例。**

角色与任务

角色与任务是最高指令，用简短的话唤醒模型在SQL生成方面的能力。

具体prompt：

```
# 角色与任务
你是专业的数据分析师，根据用户需求生成符合规范的SuperSQL代码或口径定义。
```

核心原则

核心原则是模型执行任务时必须遵守的最高准则，权重等级仅次于角色与任务。设定核心原则的目的是直接解决实践中的高频问题，例如字段名混淆、表名错误、SQL语法错误等。

具体prompt：

```
# 核心原则

### 字段使用规范
* 使用数据字段时，必须严格依据其定义口径，仅使用已明确声明的字段，禁止使用任何未定义的字段；同时，不同表
* 示例1 ：消费流水表的口径中未定义作者id字段`puin`，需通过已定义的`content_id`关联内容维表来获取。
* 示例2：对于“内容ID”这一业务含义，消费流水表中对应的字段名为`content_id`，而内容维表中对应的字段名则
```

```
### 最大引擎兼容性
* 严格遵守SuperSQL标准语法，禁止使用引擎专有语法特性，确保代码可在Spark、Presto、Starrocks等多引擎执
```

执行步骤

执行步骤用于约束模型按照固定流程思考。在"分析任务 -> 读取口径定义 -> 生成分析代码"这样逻辑性强的场景下，规定固定步骤能提升代码准确率和规范性。关键设计包括：先理解需求再生成代码、强制检索已有口径避免重复、每步都有明确输出便于人工检查。

具体prompt：

```
# 执行步骤

### 步骤1：任务类型识别
* 口径定义任务：定义指标/用户群体/内容范围等的标准SQL → 执行步骤2.1
* 数据分析任务：生成完整数据分析查询 → 执行步骤2.2

### 步骤2.1：口径定义生成
* 提取摘要：口径名称、功能、筛选条件、输出字段
* 确定分类：搜索`口径定义`下的多级子目录，确认口径分类和保存路径
* 生成代码：规范编写注释，使用CTE输出

### 步骤2.2：分析代码生成
* 提取摘要：任务名称、用户范围、时间范围、场景限制、数据源、分析任务、产出结构
* 查找口径：搜索`口径定义`目录，优先复用已有口径，缺失时告知用户补充
* 确定分类：搜索`分析代码`下的多级子目录，确认任务分类和保存路径
* 生成代码：规范编写注释，采用CTE组合多个口径，然后用一个查询语句输出最终结果。

### 步骤3：输出与说明
* 一句话说明代码功能和使用方法
```

输出格式

模型往往会生成很多冗余注释来表达自己的理解，因此我们设计了规范的注释和代码格式作为约束，要求模型只输出必要的内容。

具体prompt：

```
# 输出格式

### 口径定义
文件路径：`口径定义/[一级分类]/.../[口径名称].sql`
``` sql
/*
 * [口径名称]
```
```

```
 *
 * 【功能】
 *   - [一句话简要描述功能，如有其他重要信息也放在这里]
 *
 * 【使用方法】
 *   - [需要关联什么字段来引用此CTE]
 *
 * 【筛选条件】
 *   - ${参数1}：[定义]，格式如 [格式说明]（[数据类型]）
 *   - ${参数2}：[定义]，格式如 [格式说明]（[数据类型]）
 *
 * 【输出字段】
 *   - [字段1]：[定义]（[数据类型]）
 *   - [字段2]：[定义]（[数据类型]）
 */

cte_name AS (
    SELECT
   field1,
   field2
    FROM table_name
    WHERE condition
)
```

分析代码
* 一般文件路径：`分析代码/[一级分类]/.../[任务名称]_[YYYYMMDD].sql`
* 实验分析文件路径：`分析代码/实验/[实验名称]_[最小实验桶号]-[最大实验桶号]/[任务名称]_[YYYYMMDD].sq

``` sql
/*
 * [分析任务名称]
 *
 * 【用户范围】
 *   - [用户圈定条件]
 *
 * 【时间范围】
 *   - [具体日期或时间周期]
 *
 * 【场景限制】
 *   - [场景说明]
 *
 * 【数据源】
 *   - [表名]（[表定义]）
 *   - [表名]（[表定义]）
 *
 * 【分析任务】
 *   - [任务1]
 *   - [任务2]
 *
 * 【产出结构】
```

```
 *   - [字段1]: [定义] ([数据类型])
 *   - [字段2]: [定义] ([数据类型])
 *
 * 【结果链接】
 *   - 待补充
 */

WITH
-- [CTE功能说明]
cte_name1 AS (
    SELECT ...
    FROM ...
    WHERE ...
),

-- [CTE功能说明]
cte_name2 AS (
    SELECT ...
    FROM ...
    WHERE ...
)

-- 结果输出（不要输出其他注释）
SELECT `[中文字段1]`, `[中文字段2]` -- 使用``而非''
FROM ...
WHERE ...
``
```

生成案例

提供具体案例是提升准确率的有效手段。我们在prompt中增加了口径定义生成和分析代码生成的案例，后者包含详细的思考过程，让模型按照既定要求思考。

具体prompt：

```
# 生成案例


### 案例：口径定义生成
输入：
```
帮我增加无消费人群的口径定义，参考sql如下：
[此处省略sql]```
输出：`口径定义/用户/无消费用户.sql`
```sql
/*
 * 无消费用户群体
 *
 * 【功能】
```

```
 *   - 筛选指定周期内无消费行为的用户群体
 *
 * 【使用方法】
 *   - 关联qimei36字段
 *
 * 【筛选条件】
 *   - ${ds}: 查询日期，格式如 YYYYMMDD（STRING）
 *
 * 【输出字段】
 *   - qimei36: 用户唯一标识，来源于user_id字段（STRING）
 *   - ds: 时间分区，YYYYMMDD格式（STRING）
 */

[此处省略sql]```

### 案例：分析代码生成
输入：
```
帮我写一个数据分析sql。
需求：非一线用户美女垂类多样性指标变化；
内容：内容建联品类为美女的所有内容；
用户：非一线用户；
时间：10.2对比9.25；
指标：圈定内容下的各美女风格VV、VV占比变化。部分美女内容没有风格Tag，需要使用默认值。
```
输出：
`<搜索“口径定义/用户/一线用户”：成功>`
``` sql
/*
 * [相关注释]
 */

[此处省略sql]```
`<搜索“口径定义/内容/内容建联品类”：成功>`
``` sql
/*
 * [相关注释]
 */

[此处省略sql]```
`<搜索“口径定义/内容/美女风格tag”口径定义：成功>`
``` sql
/*
 * [相关注释]
 */

-- 提取内容Tag信息
[此处省略sql]
-- 风格Tag解析（将tags_name按逗号分割并过滤目标风格）
[此处省略sql]```
```
```

`<搜索“口径定义/消费/VV”：失败>`
已了解任务所需口径，未找到VV的定义，请用户补充。
`<用户回复并补充定义>`
`<新建“口径定义/消费/天级别VV.sql”文件：成功>`
``` sql
/*
 * [相关注释]
 */

[此处省略sql]```
已获取到所有需要的口径，下面编写sql文件。
`<新建“分析代码/用户/非一线用户/美女垂类多样性指标变化_20251016.sql”文件：成功>`
```sql
/*
 * 非一线用户美女垂类多样性指标变化分析
 *
 * 【用户范围】
 *   - 非一线城市用户
 *
 * 【时间范围】
 *   - 2025年10月2日 vs 2025年9月25日
 *
 * 【场景限制】
 *   - 无
 *
 * 【数据源】
 *   - database.table（用户画像表）
 *   - database.table（内容账号映射表）
 *   - database.table（账号基本信息表）
 *   - database.table（VV聚合表）
 *
 * 【分析任务】
 *   - 用户识别：筛选非一线城市用户
 *   - 内容筛选：建联品类为美女的内容
 *   - 指标计算：美女内容下风格Tag VV数、VV占比变化
 *
 * 【产出结构】
 *   - 风格Tag VV数、VV占比、变化率
 *
 * 【结果链接】
 *   - 待补充
 */

WITH
-- 非一线城市用户
vv_non_first_tier_users AS (
    -- 使用已有口径定义
),

-- 美女建联品类内容映射
```
```

```
vv_beauty_content AS (
    -- 使用已有口径定义
),

-- 内容风格Tag信息
vv_content_tags AS (
    -- 使用已有口径定义
),

-- 风格Tag解析（将tags_name按逗号分割并过滤目标风格）
vv_content_style_tags AS (
    -- 使用已有口径定义
),

-- VV数据
vv_analysis_data AS (
    [此处省略sql]

),

-- 非一线用户美女内容风格Tag消费数据
vv_beauty_style_consumption AS (
    [此处省略sql]
),

-- 风格Tag VV统计
style_tag_vv_stats AS (
    SELECT date_key,
 style_tag,
 SUM(vv) as total_vv
    FROM vv_beauty_style_consumption
    GROUP BY date_key, style_tag
),

-- 总VV统计
total_vv_stats AS (
    [此处省略sql]

)

-- 结果输出
[此处省略sql]```
```

通过精心设计的Agent和Prompt，我们成功实现了SQL代码的自动生成，**将编码时间从2到3小时缩短到10分钟左右**。在简单和中等复杂的场景下，生成代码的准确率能达到80%以上，这个成果已经超出了我们的预期。

然而，我们很快意识到一个关键问题：虽然代码生成很快，但数据分析工作的核心价值不在于速度，而在于准确性。即使是小概率的错误，也可能导致严重的业务决策失误。因此，在享受

AI带来的效率提升的同时，我们必须建立一套严格的质量保障体系，确保每一个生成的代码、每一份数据分析结果都经得起推敲。这就引出了知识闭环的第四个环节：让知识可信赖。



腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

690篇原创内容

公众号

6.质量保障 - 让知识可信赖

代码生成后，需要手动复制到wedata等平台执行并验证结果。考虑到数据安全问题，我们不能让Agent直接访问数据（司内的数据Agent如Dola等则可以不担心这个问题）。

AI生成的代码准确率虽然较高，但即使是小概率的错误也可能导致严重的业务决策失误。数据分析结果直接影响业务决策，容错率接近0，因此必须建立多层验证机制。以下是四种验证方法的具体实践。

6.1.业务合理性判断（最重要）

在各种验证方法中，最重要的是对核心业务指标应具备基本认知，如典型数值范围或分布规律（例如优质账号占比通常在x%左右、点击率（CTR）的合理区间等）。以下是我们数据分析时参考的核心业务指标数据图。

信息流收入

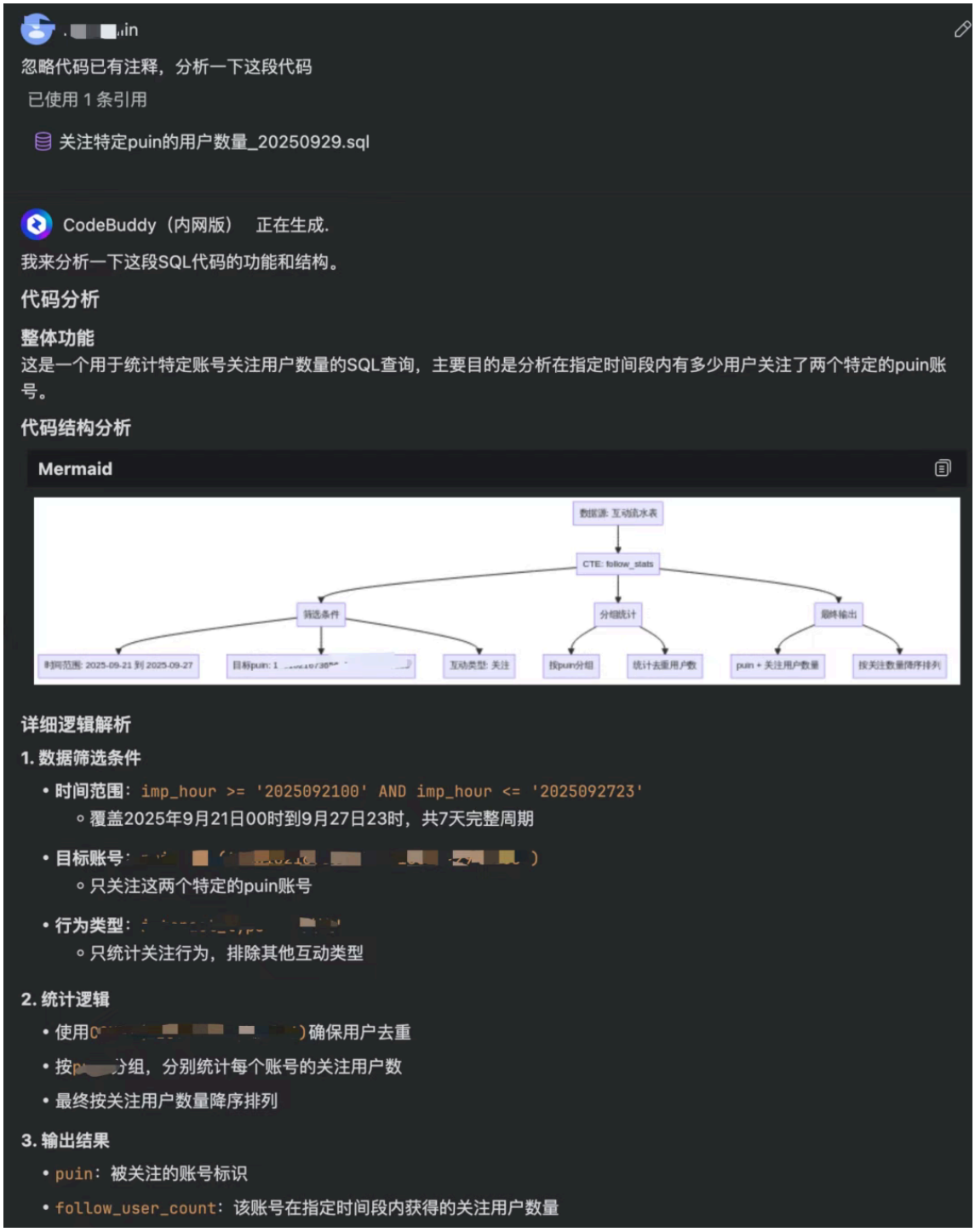
20250926 指标	当前值	日环比	周同比
QB信息流收入(单日)(万)			
主feed收入（含dsp）			
浮层收入（含dsp）			
广告总adload(含dsp，宽)			
广告总adload(含dsp，严)			
主feed adload(含dsp，宽)			
主feed adload(含dsp，严)			
浮层 adload(含dsp，宽)			
浮层 adload(含dsp，严)			
eCPM			
主feed ecpm			
浮层 ecpm			
资讯DAU(万)			
大盘DAU(万)			
PUSH DAU(万)			
短小视频VV（亿）			
信息流总时长(亿分)			
视频时长(亿分)			
主feeds视频渗透率			
视频渗透率			

6.2.AI逻辑复查

在完成SQL代码编写后，我们可以将已完成的SQL脚本交由CodeBuddy进行逻辑解析与流程图生成，通过AI的视角来辅助审查代码的逻辑结构是否合理。在使用这个方法时，我们建议在CodeBuddy中**新开一个会话窗口**，这样可以避免之前生成代码时的上下文信息对AI的判断产生干扰，从而获得更加客观的评价结果。

在实践中我们发现，**提问的方式对AI反馈的质量有很大影响**，使用"请分析这段SQL的执行逻辑"这样明确具体的提问方式，比简单地问"这段代码对吗"能够得到更有价值的反馈，因为前者会促使AI详细解释代码的执行流程和潜在问题。此外，我们还可以要求AI生成**流程图来直观展示SQL的执行逻辑**，通过可视化的方式更容易发现逻辑上的缺陷或者可以优化的地方。

AI将返回如图所示的输出。根据这段分析，可以基本判断出代码执行逻辑和数据分析需求一致。



6.3.交叉验证

对于同一批数据在不同维度下的统计结果，可通过关键指标（如总数、总和、平均值等）进行交叉核对。例如，各分项加总后应与全量汇总结果一致，从而确保统计逻辑正确。

典型应用场景：

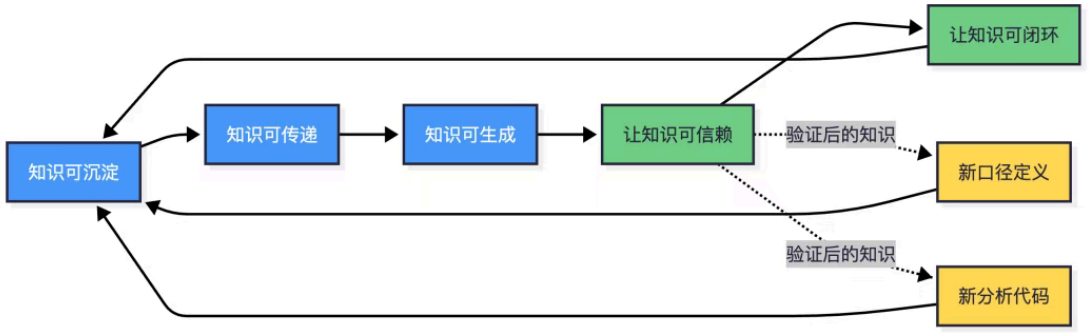
- 分城市统计的用户数之和 = 总用户数
- 各品类消费次数之和 = 总消费次数

6.4.抽样核算

针对非维度聚合类查询，可从随机抽取部分样本case，验证计算逻辑与过滤条件是否准确反映业务意图。

7.规范版本管理与归档 - 让知识可闭环

前面四个环节构建了从知识沉淀到AI生成再到质量验证的完整链路，但如果到此为止，这只是一个单向的流程。真正让这套体系产生持续价值的关键在于：经过验证的新知识能够重新回流到知识库，形成一个自我进化的闭环体系。每一次数据分析任务都不再是孤立的工作，而是在为知识库添砖加瓦，让后续的任务变得更加高效。这就是知识闭环的核心价值所在。



闭环的本质是让知识产生复利效应。当我们完成一次数据分析任务并验证通过后，这次任务中提炼出的新口径定义、优化后的SQL代码，都会通过规范化的方式沉淀回Git仓库。这些新增的知识会在下一次=任务中被AI检索和复用，从而提高生成代码的准确率。随着时间推移，知识库越来越丰富，AI生成的代码质量越来越高，团队的整体效率也在持续提升。这就形成了"**用得越多，越好用**"的正向循环，让整个体系具备了自我进化的能力。

7.1.CodeBuddy版本控制 - 让变更可追溯

Git版本控制是整个闭环体系的基础设施，每一次代码提交都是一次知识的沉淀。我们要求团队成员提交代码时必须遵循规范的commit message格式，清晰说明提交目的，比如"feat: 新增一线城市用户口径定义"或"fix: 修复浮层场景判断逻辑错误"。

7.2.规范归档 - 让结果可查找

为了让代码在几个月甚至几年后依然能被快速找到，我们从文件命名和结果记录两个维度建立了规范化的归档机制。在文件命名方面，所有分析代码的文件名必须遵循"**[任务名称][YYYYMMDD].sql**"的格式，实验分析代码的目录名必须包含实验桶号，格式为**"[实验名称][最小实验桶号]-[最大实验桶号]"**。在结果记录方面，我们要求在每个SQL文件的注释中记录分析结果的文档链接，建立代码与文档的双向关联，这样无论从代码还是文档出发，都能在30秒内找到对应的内容。

经过多层验证的SQL代码和口径定义需要及时沉淀到Git仓库，这样才能让AI在后续任务中学习和复用。我们的具体做法是：每次数据分析任务完成并验证通过后，通过Pull Request提交代码到Git仓库，代码审查时重点检查三项内容：新的口径定义是否已提取到"口径定义"目录

并添加规范注释，可复用的SQL逻辑是否已整理为独立文件便于后续参考；与此同时，当我们发现某类需求的生成效果不理想时，会将验证通过的优质代码案例补充到Agent的Prompt中作为示例，或者在Prompt中增加针对性的技巧说明，比如"处理时间字段时注意格式统一"、"多表关联时优先使用小表驱动大表"等。通过这种方式，Git仓库中的代码越来越丰富，Agent的Prompt越来越精准，形成了双向增强的效果。

8.2.闭环带来的实际价值

经过2个月的实践，知识闭环给我们团队带来了显著的改变，这些改变不仅体现在效率数据上，更体现在工作方式的根本转变上。

从**知识库的增长**来看，我们的知识库的口径已经增长到了39个常见口径， 124个分析任务。更重要的是，这些知识的复用率非常高，新增的口径定义平均每个被复用5次以上，这意味着每次知识沉淀都在为后续的多个任务节省时间。

从**团队效率**来看，单个数据分析任务的平均耗时**从4小时缩短到1小时**。更重要的是，团队成员**从重复性的代码编写工作中解放出来，有更多时间思考业务问题和优化分析方法**，工作的价值感和满意度都有明显提升。

从**协作模式**来看，我们实现了**从单人负责到多人协作的转变**，现在有数位同学可以独立完成数据分析任务，工作压力分散，不再出现因为某个人休假导致工作停滞的情况。知识传承风险也大大降低，即使有人员变动，新人也能通过知识库快速上手，不会出现知识断层的问题。

知识闭环的核心价值：让知识产生复利效应，让团队能力持续增长，让**工作方式从"重复劳动"转变为"知识积累"。**



腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

690篇原创内容

公众号

8.完整案例：从需求到交付的全流程

某天，我正在处理手头的其他事情，突然收到需求："帮跑份数据，需要筛选在2025年9月21日至27日期间关注过特定puin（xxx）的一线城市用户，查询他们在浮层场景的内容消费历史。需要召回策略号、cid、puin、标题、一级品类及URL，两个puin分开查询，每个puin抽样10个用户就行。"

看到这个需求，我的第一反应是头疼。这种跨多个数据源、涉及多个口径定义的分析任务，在以前至少需要半天时间。但现在，我打开了IDE，准备用我们团队新搭建的AI辅助数据分析体系来试试。

8.1.重构前：传统数据分析流程

如果是在几个月前，接到这样的需求我会立刻感到头疼。我需要先打开收藏夹里那一堆文档，开始漫长的查找口径定义之旅："关注过puin"对应哪张表的哪个字段来着？"一线城市"的具体范围是哪些城市？"浮层场景"的pgid取值是多少？"内容消费"的行为定义又是什么？这些问题需要我在各种文档之间来回切换，有时候文档里也找不到答案，还得去翻历史代码或者问同事。光是这个查找过程，顺利的话也要30分钟，不顺利可能要折腾一个小时。

找到所有口径定义后，接下来就是最痛苦的编码环节。我需要打开SQL编辑器，开始一个字段一个字段地敲代码。先写用户关注行为的筛选逻辑，然后关联用户画像表筛选一线城市，再关联播放流水表获取消费数据，最后关联内容维度表补充内容信息。每写一个JOIN，我都要仔细检查关联条件是否正确，生怕出现笛卡尔积导致数据爆炸。写着写着还要处理各种边界条件：时间字段格式要统一、空值要过滤、字段别名要规范。这个过程通常需要2到3个小时，而且写完之后心里还是没底，不知道逻辑对不对。

代码写完提交到WeData运行后，我会盯着屏幕等待结果。结果出来了，但这才是另一个挑战的开始：我需要验证这些数据是否准确。用户数看起来合理吗？消费数据有没有异常值？各个维度的数据能对得上吗？我会打开Excel，手动抽查几个用户的数据，再和已有的报表做对比。这个验证过程又要花掉30分钟左右，而且总是提心吊胆，担心哪里出了问题。

最后，我还要做记录归档的工作。把SQL代码复制到本地文件夹保存，给文件起个能看懂的名字，把结果链接记录到腾讯文档里，再在工作日志里简单写几句。这些琐碎的工作虽然只需要10分钟，但每次都要做，非常繁琐。整个流程下来，顺利的话要3到4个小时，如果中间遇到问题需要反复调试，可能要折腾一整天。更让人沮丧的是，下次遇到类似需求，这些工作还要重新来一遍，完全是重复劳动。

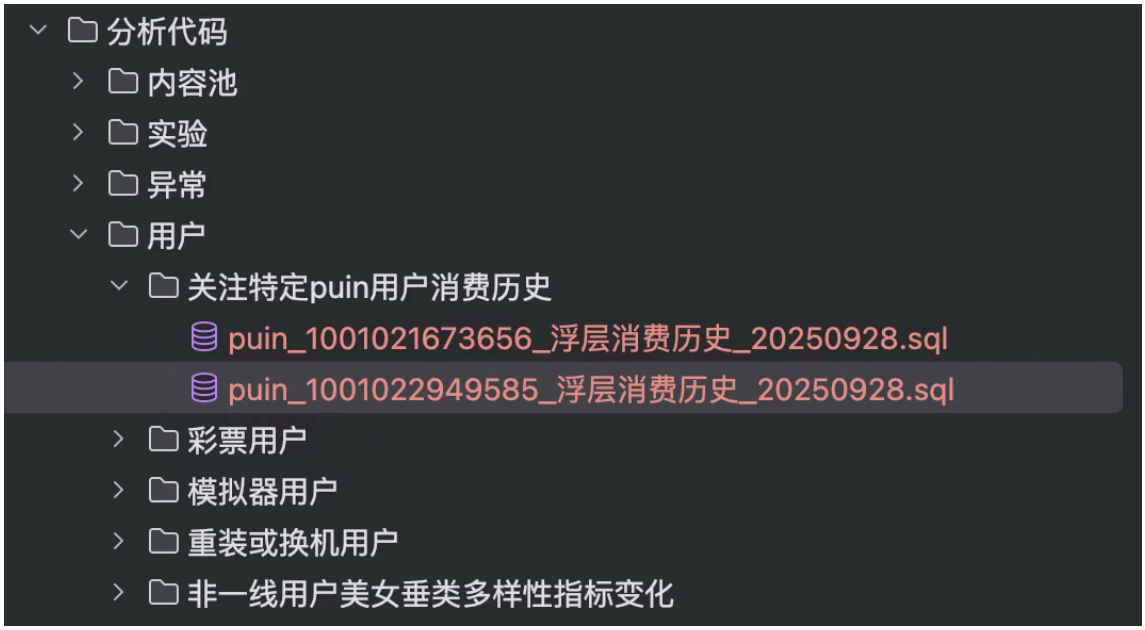
8.2.重构后：AI辅助数据分析流程

现在，让我展示一下新流程是怎么做的。

我打开IDE，点击CodeBuddy的"SQL大王"Agent，直接把需求粘贴进去："筛选在2025年9月21日至27日期间关注过puin的一线城市用户，查询他们在浮层场景的内容消费历史。需要召回策略号、cid、puin、标题、一级品类及URL。两个puin分开查询，每个puin抽样10个用户。"发送后，AI开始自动工作：在代码仓库里搜索"关注puin"相关的口径定义，找到"一线城市"的定义文件，检索"浮层场景"的判断逻辑，查找"内容消费"的数据源。不到2分钟，AI就把所有相关口径都找到了，并且给我列出了每个口径的定义和使用方法。



接下来AI开始生成代码。我看着屏幕上一行行代码自动出现，整个过程就像有个经验丰富的同事在帮我写代码一样。AI生成的代码完全遵循我们团队的编码规范：每个CTE都有清晰的业务含义，使用的是标准的CTE格式而不是嵌套子查询，文件头部有完整的注释说明任务目标和数据来源，每个处理步骤都有简洁的注释，输出字段都加上了中文别名方便理解。更让我惊喜的是，AI还自动建议了规范的文件命名"puin关注用户浮层消费分析_20250928.sql"，并提示我应该把文件存放在"分析代码/用户/"目录下，这样以后需要查找时可以快速定位。整个代码生成过程不到10分钟，我快速浏览了一遍代码，发现逻辑基本正确，有时甚至不需要微调。



下面是生成的代码片段:

```
/*
 * 关注puin_1001021673656用户浮层消费历史分析
 *
 * 【用户范围】
 *   - 目标用户：2025年9月21日-9月27日期间关注过puin 的用户
 *   - 地域限制：一线城市
 *   - 抽样规则：随机抽取10个用户
 *
 * 【时间范围】
 *   - 关注行为周期：2025年9月21日00时-9月27日23时
 *   - 消费行为周期：2025年9月21日00时-9月27日23时
 *
 * 【场景限制】
 *   - 消费场景：浮层场景 *
 * 【数据源】
 *   - database.table（互动流水表）
 *   - database.table（用户画像表）
 *   - database.table（视频播放流水表）
 *   - database.table（内容维度表）
 *
 * 【分析任务】
 *   - 用户识别：筛选关注puin 1001021673656且为一线城市的用户
 *   - 随机抽样：随机抽取10个用户
 *   - 消费分析：统计用户在浮层对该puin内容的消费历史记录
 *   - 内容关联：关联内容维度信息获取完整内容属性
 *
 * 【产出结构】
 *   - qimei36：用户标识（STRING）
 *   - strategy_id：召回策略号（STRING）
 *   - cid：内容ID（STRING）
```

```
 *   - puin: 内容创作者puin (STRING)
 *   - title: 内容标题 (STRING)
 *   - fst_chann_cn: 一级品类 (STRING)
 *   - url: 内容链接 (STRING)
 *   - play_time: 播放时间 (STRING)
 *   - video_length: 视频时长 (BIGINT)
 *
 * 【结果链接】
 *   - 待补充
 */

WITH
-- 关注puin 1001021673656的用户
follow_users AS (
    SELECT DISTINCT
   qimei36,
   puin
    FROM database.table
    WHERE imp_hour >= 2025092100
   AND imp_hour <= 2025092723
   AND interact_type = '关注'
   AND puin = '1001021673656'
   AND qimei36 IS NOT NULL
   AND qimei36 != ''
),
。。。。
```

代码生成后，我没有立即提交运行，而是先进行了验证。我新开了一个CodeBuddy会话窗口，把刚才生成的SQL代码粘贴进去，让AI帮我逐步解释代码逻辑。AI很快给出了详细的分析，从数据源选择、关联逻辑、筛选条件到聚合计算，每一步都解释得很清楚。在解释过程中，AI还指出了一个潜在问题：时间字段的格式在不同表中可能不一致，建议我统一处理成相同格式再进行关联。我按照AI的建议修正了代码，确认逻辑没有问题后，将代码复制到WeData平台提交运行。在等待结果的几分钟里，我根据以往的经验快速估算了一下结果的量级，按照这个用户群体的规模和历史消费情况，预计应该会有几千个用户、上万次消费行为。结果出来后，我检查了关键指标的合理性，用户数、消费次数都在预期范围内，各维度数据也能对得上，没有出现异常的空值或者数量级偏差。虽然验证环节**仍然需要**我的业务判断和经验积累，但有了AI的辅助解释和逻辑检查，整个验证过程只用了15分钟，比以前快了一半，而且准确性更有保障。

最后是归档工作。我把验证通过的代码提交到Git仓库，在文件注释中补充了结果文档的链接，整个过程不到1分钟就完成了。因为有了规范的文件命名和Git版本控制，我知道以后如果需要回溯这次分析，可以在几秒钟内找到相关代码和结果。

	重构前	重构后
查找口径	翻阅各种文档,在收藏夹里来回切换,有时还要翻历史代码或问同事	2分钟让CodeBuddy自动检索Git仓库中的口径定义,清晰列出每个口径的定义和使用方法（口径已经在知识库的情况下）
编写代码	数小时手工编写复杂的多表关联逻辑,需要仔细检查每个JOIN条件,处理各种边界条件	10分钟让CodeBuddy生成符合团队规范的SQL代码,使用标准CTE格式,包含完整注释
验证结果	完全依赖人工检查,手动抽查用户数据,与已有报表对比,提心吊胆担心出错	引入CodeBuddy辅助进行逻辑复查,人工业务判断仍是核心,但有AI辅助解释和检查
记录归档	10分钟手动记录各种信息,复制代码到本地文件夹,给文件起名,记录链接到文档，容易遗失	1分钟借助Git版本控制完成归档,规范的文件命名和版本管理让后续查找变得简单

注意：本次案例**基于口径已存在于知识库的情况下**，若设计新口径，流程会更长。

9.展望后续改进

9.1.为什么选择CodeBuddy

在引入AI辅助编程工具时，我们对比了内部的Text2SQL框架（vanna）、公司内部数据Agent。最终选择CodeBuddy，是因为它在我们的实际场景中最合适。它能深度集成到PyCharm等日常开发环境中，可以索引和理解整个Git仓库的代码结构和历史沉淀，支持自定义Prompt将业务规则和编码规范融入生成过程，同时学习成本低。更重要的是，CodeBuddy能够很好地融入我们构建的知识闭环体系，成为连接知识沉淀和知识生成的关键桥梁，而不是一个孤立的工具。工具选型的核心不是追求"最先进"，而是找到与团队能力、业务场景、现有工作流程最匹配的方案。

9.2.后续改进方向

9.2.1.自动生成口径说明文档

现状：我们目前只维护Git代码仓库作为唯一的口径数据源，同时已经有挂载了iwiki的企微机器人用于询问口径问题。但Git代码对非技术人员来说理解门槛太高，而同时维护Git和iwiki又显得冗余，容易出现不一致的情况。这就形成了一个矛盾：技术人员觉得维护两套麻烦，非技术人员又看不懂代码。

方案：开发一个自动化脚本，在新增口径代码提交到Git时，自动触发AI读取代码并生成通俗易懂的口径说明文档（包括业务含义、计算逻辑、使用场景等），经过审核后自动同步到iwiki平台。这样既保证了Git作为单一数据源的权威性，又让非技术人员能通过iwiki或企微机器人方便地查询和理解口径。

9.2.2.打通SQL执行环节

AI生成SQL后仍需手工复制到WeData执行，流程割裂。但用claude4.5读取数据，会有一定数据安全问题，开源模型的效果又可能不足，需要进一步调研如何把两个过程衔接起来。

9.3.回顾：知识闭环的形成

回顾整个重构过程，我们实际上构建了一个完整的知识闭环体系，这个体系由五个相互关联的环节组成，每个环节都解决了传统工作方式中的关键痛点。首先是**让知识可沉淀**，我们通过规范化的文档和代码管理，将散落在各处的口径定义、业务规则和技术经验体系化地保存下来，避免了知识随着人员流动而流失。接着是**让知识可传递**，通过Git仓库和iwiki文档的结合，团队成员可以随时查阅和学习前人的经验，新人上手时间从原来的两周缩短到三天。然后是**让知识可生成**，AI基于沉淀的知识库理解业务逻辑和编码规范，自动生成高质量的代码，将重复性工作的效率提升了85%。第四步是**让知识可信赖**，通过AI辅助验证、业务经验估算和多维度交叉检查，确保生成的代码逻辑正确、结果可靠，让团队敢于放心使用AI生成的代码。最后是**让知识可闭环**，每次数据分析产生的SQL代码、踩坑经验和优化技巧都会经过验证后重新沉淀到知识库中，这些新知识又会反过来提升AI的生成能力，形成一个正向循环的飞轮效应。这个闭环体系的核心价值在于，知识库越丰富，AI生成的代码就越准确，团队的整体效率就会持续提升，而不是停留在某个固定水平。更重要的是，这个体系**让知识真正成为了团队的资产，而不再是个人的私有财产，每个人的贡献都会让整个团队受益**，这种协同效应带来的价值远超单纯的效率提升。



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
690篇原创内容
公众号



阅读 7599

留言

写留言

还没有留言，来写首评